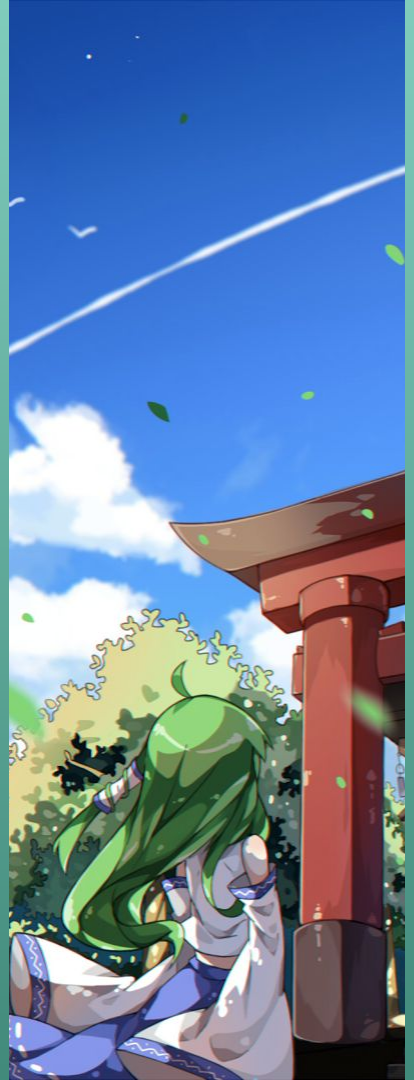


touhou miko senjou

東方巫女戦場 ～ Double Prayer Duel

Powered by Curtain Engine

Nathan Ng Patrick Ng Cyrus Villaruz

Background

Background

Touhou Project

- **東方巫女戦場 ～ Double Prayer Duel** is a Touhou Project fangame
- Touhou Project: A collection of official and fan media, including games, anime, music, etc.
  - Games are centered around the fantasy land of Gensokyo
  - You may have seen some fan games on Steam and console storefronts (Touhou Luna Nights, Touhou Lost Word)

Background

Gameplay

- Official mainline games have 2D bullet hell gameplay
- Traditional arcade-style scoring and lives
- Enemies and bullet patterns are pre-scripted
- Player has a tiny hitbox to allow them to weave between gaps

Background

Story

- The shrine maiden of the Hakurei Shrine, Reimu, has won the lottery and amassed a great fortune, finally bringing wealth to the formerly bankrupt shrine
- Sanae of the Moriya Shrine learns of this and decides to take shrine rivalry seriously
- Reimu is left to defend the Hakurei Shrine from an onslaught of Moriya Shrine-affiliated fairies before confronting Sanae herself

Components and Systems

System / Enhancement

Sprite

A new **pivot offset** field allows for sprites to be offset relative to their actual position.

Before

After

```
53  struct Sprite {
54      SDL_Texture* texture = nullptr;
55      SDL_FRect src{};
56      SDL_FRect dst{};
57      RenderLayer renderLayer = RenderLayer::World;
58      Vector2D pivotOffset = Vector2D( x: 0,  y: 0);
59      bool visible = true;
60  };
```

```
30      for (auto& entity :unique_ptr<Entity> & : entities) {
31          if (entity->hasComponent<Transform>() && entity->hasComponent<Sprite>()) {
32              auto& t :Transform & = entity->getComponent<Transform>();
33              auto& sprite = entity->getComponent<Sprite>();
34
35              // Only render sprites on the World layer.
36              if (sprite.renderLayer != RenderLayer::World) {
37                  continue;
38              }
39
40              // We are converting from world space to screen space.
41              sprite.dst.x = t.position.x - sprite.pivotOffset.x - cam.view.x;
42              sprite.dst.y = t.position.y - sprite.pivotOffset.y - cam.view.y;
43
44              // If the entity has animation, update the source rect.
45              if (entity->hasComponent<Animation>()) {
46                  auto& anim :Animation & = entity->getComponent<Animation>();
47                  sprite.src = anim.clips[anim.currentClip].frameIndices[anim.currentFrame];
48              }
49
50              TextureManager::draw(sprite.texture, &sprite.src, &sprite.dst);
51          }
52      }
```

System / Enhancement

Movement

Updates to position & rotation will also update the **position & rotation of children**.

Before

After

```
struct Children {
    std::vector<Entity*> children{};
};
```

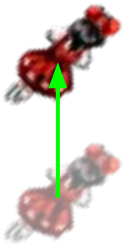
```
// Vector2D needs an operator function to multiply a float.
Vector2D velocityVector = directionVector * v.currentSpeed;

// Update position.
t.position += velocityVector * dt;

// Update position of children Transforms.
if (entity->hasComponent<Children>()) {
    auto&[children vector<Entity *> ] = entity->getComponent<Children>();

    for (const auto& child Entity * const & : children) {
        if (child->hasComponent<Transform>()) {
            auto& childTransform = child->getComponent<Transform>();
            childTransform.position += t.position - t.oldPosition;
        }
    }
}
```

System / Enhancement

Velocity

A **local space** setting allows velocity to be influenced by rotation.

Before



After

```cpp
// Direction and speed.
struct Velocity {
    Vector2D direction{};
    float baseSpeed{};

    // If true, the direction is influenced by Transform.rotation.
    bool isLocalSpace{};

    float currentSpeed = baseSpeed;
};
```

```cpp
        Vector2D directionVector = v.direction;

        // Normalize.
        directionVector.normalize();

        // If local space is on, add rotation to the direction.
        if (v.isLocalSpace) {
            float rotationRadians = std::atan2(directionVector.y, directionVector.x);
            rotationRadians += t.rotation * (std::numbers::pi / 180.0f);
            std::fmod( x: rotationRadians,  y: 360.0f);

            directionVector.x = std::cos(rotationRadians);
            directionVector.y = std::sin(rotationRadians);
        }

        // Vector2D needs an operator function to multiply a float.
        Vector2D velocityVector = directionVector * v.currentSpeed;

        // Update position.
        t.position += velocityVector * dt;
```

Component / Transform

AngularVelocity

Modifies the entity's **rotation over time** .

```
// Rotation over time, which modifies the rotation property of the Transform.
struct AngularVelocity {
    float rotationOverTime{};
};
```

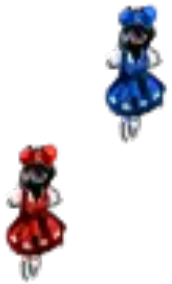
```
static void updateRotationBasedOnAngularVelocity(float dt, std::unique_ptr<Entity> &entity) {
    auto& t :Transform & = entity->getComponent<Transform>();
    auto& v :AngularVelocity & = entity->getComponent<AngularVelocity>();

    float rotationAmountDegrees = v.rotationOverTime * dt;

    t.rotation = std::fmod( x: t.rotation + rotationAmountDegrees, y: 360.0f);
```

Component / Transform

LookAtRotator

Modifies the **rotation** so that the entity is **always pointing at a specified Transform** .

Before



After

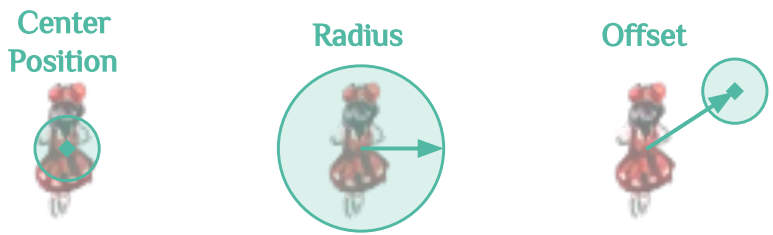
```
// Sets the rotation property of the Transform to point toward a different target Transform.
struct LookAtRotator {
    // The target so that the rotator will point local space DOWN toward the target.
    Transform& target;

    // Offset to the target in degrees.
    float offsetDegrees{};
};
```

```
    static void updateRotationBasedOnTarget(std::unique_ptr<Entity> &entity) {
        auto& t Transform& = entity->getComponent<Transform>();
        auto& r LookAtRotator& = entity->getComponent<LookAtRotator>();

        const Vector2D displacement = r.target.position - t.position;

        float oldRotation = t.rotation;

        t.rotation = std::fmod( x: std::atan2(displacement.y, displacement.x) * (180 / std::numbers::pi) +
            r.offsetDegrees - 90.0f, y: 360.0f);

```

Component / Collision

CircleCollider

Represents a circle-shaped collider with a **center position**, **radius**, and **offset**.



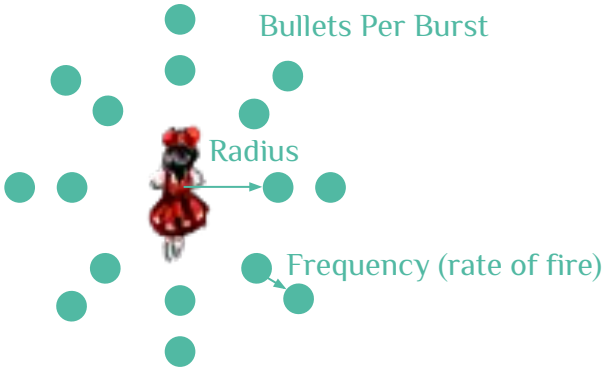

```
struct CircleCollider {
    std::string tag;
    float radius{}; // Radius of the circle used for detecting collisions.
    Vector2D centerPosition{}; // Position of the collider (this is the actual position, updated by collision system).
    Vector2D offset{}; // Offset for collider center relative to the entity's transform.
    bool enabled = true;
};
```

```
// Circle collision.
bool Collision::Circle(const CircleCollider& colA, const CircleCollider& colB) {
    auto& colAPosition = colA.centerPosition;
    auto& colBPosition = colB.centerPosition;
    float distanceBetweenColliders = (colBPosition - colAPosition).length();

    return distanceBetweenColliders <= colA.radius + colB.radius;
}

// AABB x Circle collision.
bool Collision::AABBCircle(const SDL_FRect& colA, const Vector2D& colBCenterPosition, const float& colBRadius) {
    // Convert colB position and radius into an FRect square approximation.
    const SDL_FRect& colB = SDL_FRect{
        x: colBCenterPosition.x - colBRadius,  y: colBCenterPosition.y - colBRadius,  w: colBRadius * 2,  h: colBRadius * 2};

    return AABB(colA, colB);
}
```

Component / Danmaku

RadialSpawner

Spawns projectiles in a **circular pattern** around the entity with various settings.




```cpp
struct RadialSpawner {
    // Whether to emit projectiles.
    bool isActive{};

    // The frequency (sec) that one burst of bullets is emitted.
    float frequency{};

    // Bullet emission speed.
    float bulletEmissionSpeed{};

    // Bullet emission angular velocity (rotation over time).
    float bulletEmissionAngularVelocity{};

    // Inner radius of bullet pattern emitters.
    float radius{};

    // Number of bullets in each burst.
    int bulletsPerBurst{};

    // Invoked when spawning, once per bullet, with param direction.
    std::function<void(Vector2D)> spawnCallback{};

    // The actual spawn countdown, which triggers spawn when it hits zero then resets.
    float spawnTimer{frequency};
};
```

```cpp
if (spawner.spawnTimer <= 0) {
    spawner.spawnTimer += spawner.frequency;

    float angleBetweenBullets = 360.0f / spawner.bulletsPerBurst;

    float currentEmissionAngle = transform.rotation;
    // std::cout << "Spawner Emission Angle: " << currentEmissionAngle << std::endl;

    for (int i = 0; i < spawner.bulletsPerBurst; i++) {
        spawner.spawnCallback(Vector2D(currentEmissionAngle));

        currentEmissionAngle += angleBetweenBullets;
    }
}
```

Component / Danmaku

LinearSpawner

Spawns projectiles in a **line or fan pattern** around the entity with various settings.

```
struct LinearSpawner {
    // Whether to emit projectiles.
    bool isActive{};

    // Whether bullets radiate from emitter center after emission.
    bool isFanPattern;

    // Bullet emission speed.
    float bulletEmissionSpeed{};

    // Bullet speed multiplier relative to the distance between bullet and emitter center.
    float bulletEmissionSpeedMultiplier{};

    // List of positions for bullets to emit in the burst relative to emitter center.
    std::vector<Vector2D> bulletPositions{};

    // The frequency (sec) that one burst of bullets is emitted
    float frequency{};

    // Invoked when spawning, once per bullet, with params global position, direction, speed.
    std::function<void(Vector2D, Vector2D, float)> spawnCallback{};

    // The actual spawn countdown, which triggers spawn when it hits zero then resets.
    float spawnTimer{frequency};
};
```

Line Pattern

Fan Pattern

```
if (spawner.spawnTimer <= 0) {
    spawner.spawnTimer += spawner.frequency;

    for (int i = 0; i < spawner.bulletPositions.size(); i++) {
        auto& localPosition Vector2D & = spawner.bulletPositions[i];

        // Rotate the local position according to the spawner's rotation.
        float angleRadians = std::atan2(localPosition.y, localPosition.x) +
            transform.rotation * (std::numbers::pi / 180.0f);
        float radius = localPosition.length();

        auto rotatedLocalPosition = Vector2D( x: std::cos(angleRadians) * radius,  y: std::sin(angleRadians) * radius);
        const Vector2D spawnPosition = transform.position + rotatedLocalPosition;
        Vector2D displacement = -rotatedLocalPosition;

        float rotation;

        if (spawner.isFanPattern) {
            rotation = std::atan2(displacement.y, displacement.x) + std::numbers::pi / 2;
        }
        else {
            rotation = transform.rotation / (180 / std::numbers::pi);
        }

        const float spawnSpeed = spawner.bulletEmissionSpeed +
            displacement.length() * spawner.bulletEmissionSpeedMultiplier;

        const auto direction = Vector2D( degrees: rotation * (180 / std::numbers::pi));

        spawner.spawnCallback(spawnPosition, direction, spawnSpeed);
    }
}
```

Component / Sequencing

Timeline

Calls **specific functions** at **specific times** .

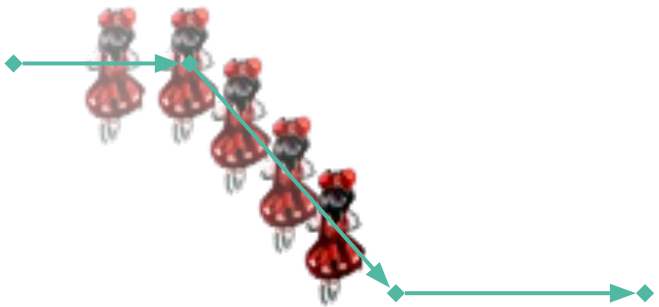
```cpp
// Controls pre-scripted events at specific times.
struct Timeline {
    float currentTime = 0;

    // Timeline elements have a float (time to trigger the action) and function (the action to trigger).
    std::vector<std::pair<float, std::function<void()>>> timeline();
};
```

```cpp
auto &[currentTime :float , timeline :vector<pair<float,function<void()>>> ] = entity->getComponent<Timeline>();

// Skip this timeline since it has already finished playing. Assuming actions are ordered by time.
if (currentTime > timeline.back().first) {
    continue;
}

const float timeBeforeThisFrame = currentTime;
currentTime += dt;

const float differenceBetweenFrames = currentTime - timeBeforeThisFrame;

for (const auto &[timestamp :const float , action :const function<void()> ]: timeline) {
    // If this action should occur between last frame and this frame.
    if (currentTime - timestamp < differenceBetweenFrames && currentTime - timestamp >= 0) {
        action();
    }
}
```

Component / Sequencing

Path

Consists of **sequential Points** and other settings for **scripting enemy movement** .



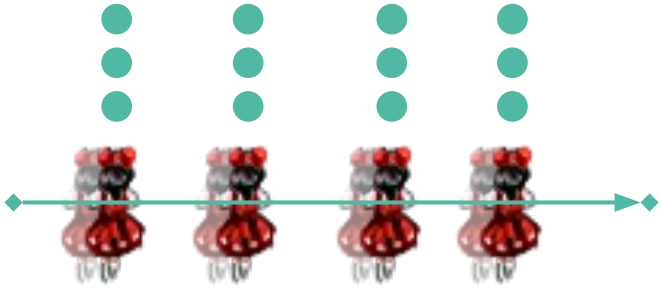
```
struct Path {
    std::vector<PathPoint> points{};
};

struct PathFollower {
    int pathId{};
    float distance{};
    float speed{};
    bool active = true;
    float currentHoverTimer = 0.0f;
    int lastPointReached = -1;
};
```

```
Vector2D PathSystem::evaluatePath(const Path& path, float distance) {
    float remainingPoints = distance;

    for (size_t i = 0; i < path.points.size() - 1; i++) {
        auto a :Vector2D = path.points[i].position;
        auto b :Vector2D = path.points[i + 1].position;

        float length = (b - a).length();

        if (remainingPoints <= length) {
            float t = remainingPoints / length;
            return a + (b - a) * t;
        }

        remainingPoints -= length;
    }

    return path.points.back().position;
}
```

System / Sequencing

Convoy

A group of enemies that **follow the same Path** and have **the same Danmaku Pattern**.



```
struct Convoy {
    EnemyType enemyType{};
    int pathId{};
    int numEnemies{};
    float speed{};
    float spawnInterval{};
    float timer = 0.0f;

    DanmakuPattern danmakuPattern{};
};
```

```
void EnemyFactory::buildEnemy(Entity &entity, World& world, const Convoy& convoyData) {
    entity.addComponent<PathFollower>(convoyData.pathId, 0.0f, convoyData.speed);
    entity.addComponent<Velocity>(Vector2D(x:0.0f, y:0.0f), 0.0f);
    entity.addComponent<EnemyTag>();

    switch (convoyData.enemyType) {
        case EnemyType::SmallBlueFairy:
        case EnemyType::SmallRedFairy:
            buildSmallFairy([&]entity, [&]world, convoyData.enemyType, convoyData.danmakuPattern);
            break;
```

System / UI

SelectableUI

Allows the player to navigate UI using **keyboard controls** .

Before

Resume Game

Quit to Title

After

Resume Game

Quit to Title

W

A S D

Enter

```
struct SelectableUI {
    std::function<void()> onPressed{};
    std::function<void()> onReleased{};
    std::function<void()> onSelect{};
    bool selected = false;

    // The selectable UI elements are a doubly linked list.
    Entity* next = nullptr;
    Entity* previous = nullptr;
};
```

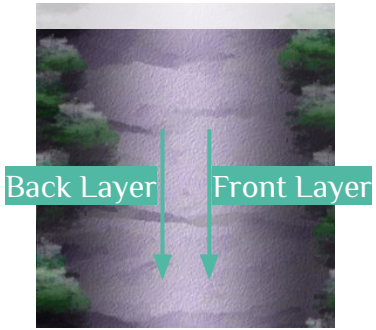
```
// UI interaction with keyboard via events.
if (currentlySelectedUI && event.type == SDL_EVENT_KEY_DOWN) {
    switch (event.key.key) {
        case SDLK_Z:
        case SDLK_RETURN:
            world.getEventManager().emit(UIInteractionEvent{currentlySelectedEntity, UIInteractionState::Pressed});
            break;
        case SDLK_W:
        case SDLK_UP:
            world.getEventManager().emit(UIInteractionEvent{currentlySelectedEntity, UIInteractionState::Released});
            world.getEventManager().emit(UIInteractionEvent{currentlySelectedUI->previous, UIInteractionState::Selected});
            break;
        case SDLK_S:
        case SDLK_DOWN:
            world.getEventManager().emit(UIInteractionEvent{currentlySelectedEntity, UIInteractionState::Released});
            world.getEventManager().emit(UIInteractionEvent{currentlySelectedUI->next, UIInteractionState::Selected});
            break;
        default:
            break;
    }
}
```

System / Graphics

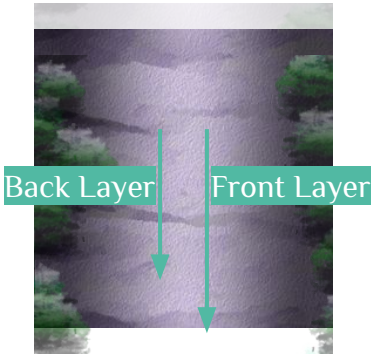
StageBackground

Renders a **repeating background** with **parallax scrolling**.

Before



After




```
struct StageBackground {
    float scrollSpeedY = 100.0f;
    float offsetY = 0.0f;
    SDL_Texture* texture{};
};
```

```
auto& stageBackground = entity->getComponent<StageBackground>();
auto& transform = entity->getComponent<Transform>();
auto& sprite = entity->getComponent<Sprite>();

// Track how far the background should have moved down the screen.
// Note that the MovementSystem is responsible for doing the actual movement.
float offset = stageBackground.scrollSpeedY * dt;
stageBackground.offsetY += offset;

// Move background back to the top when it passes its original height to loop it.
if (stageBackground.offsetY >= sprite.dst.h) {
    transform.position.y -= sprite.dst.h;
    stageBackground.offsetY -= sprite.dst.h;
}
```

System / Debug

DebugRender

Toggles on or off **collider bounds visuals** to assist with debugging collisions.

Before

After

```
    for (auto& entity : const unique_ptr<Entity> & : entities) {
        if (entity->hasComponent<RectCollider>()) {
            auto& c : RectCollider & = entity->getComponent<RectCollider>();

            SDL_Texture* tex = TextureManager::load( path: "../asset/tileset.png");
            SDL_FRect colSrc { .x: 32, .y: 32, .w: 32, .h: 32};

            TextureManager::draw(tex, &colSrc, dst: &c.rect);
        }
        if (entity->hasComponent<CircleCollider>()) {
            auto& c : CircleCollider & = entity->getComponent<CircleCollider>();

            SDL_Texture* tex = TextureManager::load( path: "../asset/ui/debug-circle.png");
            SDL_FRect colSrc { .x: 0, .y: 0, .w: 32, .h: 32};

            float colliderTopLeftX = c.centerPosition.x - c.radius;
            float colliderTopLeftY = c.centerPosition.y - c.radius;

            SDL_FRect colDst {colliderTopLeftX, colliderTopLeftY, .w: c.radius * 2, .h: c.radius * 2};

            TextureManager::draw(tex, &colSrc, &colDst);
        }
```

Live Demo

Behind the Scenes

Stage Scripting

Stage Runthrough 1/3

Load XML

Our entire stage is stored in one XML file with various sections.

- A list of **preset Paths** for enemies to use
- **Waves of enemies** represented as **Convoys**
- **Danmaku Patterns** used by enemies in Convoys

```xml
<Stage>
    <Paths>
        <Path id="0" name="ZigZagLeft">
            <Point x="46" y="-30" />
            <Point x="46" y="314" />
            <Point x="366" y="462" />
            <Point x="16" y="611" />
            <Point x="366" y="760" />
            <Point x="16" y="1016" />
        </Path>
        <Path id="1" name="ZigZagRight">
            <Point x="722" y="-30" />
            <Point x="722" y="314" />
            <Point x="402" y="462" />
            <Point x="752" y="611" />
            <Point x="402" y="760" />
            <Point x="752" y="1016" />
        </Path>
        <Path id="2" name="StraightDownLeft">
            <Point x="56" y="-34" />
            <Point x="56" y="266" />
            <Point x="56" y="1016" />
        </Path>
```

```xml
    <Waves>
        <Convoy start="2.0" enemyType="SmallBlueFairy" pathId="0" numEnemies="10" interval="0.5" speed="200">
            <Pattern type="Linear" bullet="Circle" startTime="0.2" endTime="2.0"
                    frequency="0.2" bulletSpeed="400.0" isFan="true" targeted="true" speedMultiplier="1.0">
                <BulletPositions>
                    <Pos x="0" y="0" />
                </BulletPositions>
            </Pattern>
        </Convoy>
        <Convoy start="2.0" enemyType="SmallRedFairy" pathId="1" numEnemies="10" interval="0.5" speed="200">
            <Pattern type="Linear" bullet="Circle" startTime="0.2" endTime="2.0"
                    frequency="0.2" bulletSpeed="400.0" isFan="true" targeted="true" speedMultiplier="1.0">
                <BulletPositions>
                    <Pos x="0" y="0" />
                </BulletPositions>
            </Pattern>
        </Convoy>

        <Convoy start="5.0" enemyType="LargeFairy" pathId="8" numEnemies="1" speed="300">
            <Pattern type="Radial" bullet="Circle" startTime="1.0" endTime="6.0" frequency="1.0" bulletSpeed="150.0"
                    bulletsPerBurst="16" rotationSpeed="0.0" bulletAngularVel="0.0" radius="12.0"/>
        </Convoy>

        <Convoy start="15.0" enemyType="SmallBlueFairy" pathId="2" numEnemies="5" interval="1.0" speed="200" />
        <Convoy start="15.0" enemyType="SmallRedFairy" pathId="3" numEnemies="5" interval="1.0" speed="200" />
```

Stage Runthrough 2/3

Timeline

- When loading the scene, each Convoy's data **gets added to the Timeline**
- As the player plays through the stage, the **Timeline will spawn Convoys at the specified times**

```
    auto* wavesRoot XMLElement* = root->FirstChildElement( name: "Waves");
    for (auto* convoyElement = wavesRoot->FirstChildElement( name: "Convoy");
        convoyElement;
        convoyElement = convoyElement->NextSiblingElement( name: "Convoy")) {
        float startTime = convoyElement->FloatAttribute( name: "start");

        Convoy data;
        data.enemyType = stringToEnemyType( name: convoyElement->Attribute( name: "enemyType"));
        data.pathId = convoyElement->IntAttribute( name: "pathId");
        data.numEnemies = convoyElement->IntAttribute( name: "numEnemies");
        data.spawnInterval = convoyElement->FloatAttribute( name: "interval");
        data.speed = convoyElement->FloatAttribute( name: "speed");

        auto* danmakuPatternElem XMLElement* = convoyElement->FirstChildElement( name: "Pattern");
        if (danmakuPatternElem) {
            data.danmakuPattern.hasPattern = true;
            data.danmakuPattern.danmakuType = stringToDanmakuType( name: danmakuPatternElem->Attribute( name: "type"));
            data.danmakuPattern.bulletType = stringToBulletType( name: danmakuPatternElem->Attribute( name: "bullet"));
            data.danmakuPattern.startTime = danmakuPatternElem->FloatAttribute( name: "startTime");
            data.danmakuPattern.endTime = danmakuPatternElem->FloatAttribute( name: "endTime");
            data.danmakuPattern.frequency = danmakuPatternElem->FloatAttribute( name: "frequency");
            data.danmakuPattern.bulletSpeed = danmakuPatternElem->FloatAttribute( name: "bulletSpeed");

            data.danmakuPattern.bulletsPerBurst = danmakuPatternElem->IntAttribute( name: "bulletsPerBurst");
            data.danmakuPattern.rotationSpeed = danmakuPatternElem->FloatAttribute( name: "rotationSpeed");
            data.danmakuPattern.bulletAngularVel = danmakuPatternElem->FloatAttribute( name: "bulletAngularVel");
            data.danmakuPattern.radius = danmakuPatternElem->FloatAttribute( name: "radius");

            data.danmakuPattern.isFanPattern = danmakuPatternElem->BoolAttribute( name: "isFanPattern");
            data.danmakuPattern.shouldTargetPlayer = danmakuPatternElem->BoolAttribute( name: "targeted");
            data.danmakuPattern.speedMultiplier = danmakuPatternElem->FloatAttribute( name: "speedMultiplier");

            auto* bulletPosRoot XMLElement* = danmakuPatternElem->FirstChildElement( name: "BulletPositions");
            if (bulletPosRoot) {
                for (auto* pos XMLElement* = bulletPosRoot->FirstChildElement( name: "Pos");
                     pos;
                     pos = pos->NextSiblingElement( name: "Pos")) {

                    float posX = pos->FloatAttribute( name: "x");
                    float posY = pos->FloatAttribute( name: "y");
                    data.danmakuPattern.bulletPositions.emplace_back(posX, posY);
                }
            }
        }

        timelineEntity.getComponent<Timeline>().timeline.emplace_back( [&] startTime,  y: [&world, data] () ->void  {
            auto& spawner Entity& = world.createDeferredEntity();
            spawner.addComponent<Convoy>(data);
        });
    }
```

Stage Runthrough 3/3

Instantiation

- **Enemy factories** handle the instantiation of different enemy types
- **Danmaku factories** handle the instantiation of different bullet patterns
- Enemies and bullets are **destroyed after leaving the play boundaries**

```cpp
void EnemyFactory::buildSmallFairy(Entity &entity, World& world, const EnemyType& enemyType, const DanmakuPattern& danmakuPattern) {
    entity.addComponent<EnemyHealth>(100, 100);
    auto& loot = entity.addComponent<LootDropTable>();
    loot.offsets = { { x:0, y:0} };

    if (enemyType == EnemyType::SmallRedFairy) {
        initBaseFairy(&entity, animName: "redFairy", spritesheetPath: "../asset/animations/small_fairies_anim.png", size:32);
        loot.types = { SmallPower };
    } else {
        initBaseFairy(&entity, animName: "blueFairy", spritesheetPath: "../asset/animations/small_fairies_anim.png", size:32);
        loot.types = { Point };
    }

    if (danmakuPattern.shouldTargetPlayer) {
        for (auto& e unique_ptr<Entity>& : world.getEntities()) {
            if (e->hasComponent<PlayerTag>()) {
                auto& playerTransform = e->getComponent<Transform>();
                entity.addComponent<LookAtRotator>(->playerTransform, 0.0f);
                break;
            }
        }
    }

    DanmakuFactory::buildDanmaku(&entity, &world, danmakuPattern);
}
```

```cpp
void DanmakuFactory::initRadialPattern(Entity& entity, World& world, const DanmakuPattern& danmakuPattern) {
    entity.addComponent<RadialSpawner>(
        true,
        danmakuPattern.rotationSpeed,
        danmakuPattern.frequency,
        danmakuPattern.bulletSpeed,
        danmakuPattern.bulletAngularVel,
        danmakuPattern.radius,
        danmakuPattern.bulletsPerBurst,
        [&world, danmakuPattern, &entity](Vector2D direction)->void {
            if (!entity.isActive()) {
                return;
            }

            auto& transform = entity.getComponent<Transform>();
            Vector2D spawnPos = transform.position + (direction * danmakuPattern.radius);

            auto& bullet Entity& = world.createDeferredEntity();

            auto& bulletTransform = bullet.addComponent<Transform>(->spawnPos, 0.0f, 1.0f);
            bullet.addComponent<Velocity>(->direction, danmakuPattern.bulletSpeed, true);
            bullet.addComponent<AngularVelocity>(danmakuPattern.bulletAngularVel);
            bullet.addComponent<ProjectileTag>();

            SDL_Texture* tex = TextureManager::load( path: "../asset/bullet4.png");
            SDL_FRect src { x:192, y:0, w:64, h:64};
            SDL_FRect dst {spawnPos.x, spawnPos.y, w:64, h:64};
            Vector2D pivotOffset = Vector2D( x:dst.w / 2.0f, y:dst.h / 2.0f);
            bullet.addComponent<Sprite>(->tex, ->src, ->dst, RenderLayer::World, ->pivotOffset);

            Vector2D bulletSpawnPositionOffset = direction * danmakuPattern.radius;
            auto& bulletCol CircleCollider& = bullet.addComponent<CircleCollider>("projectile");
            bulletCol.centerPosition = bulletTransform.position + bulletSpawnPositionOffset;
            bulletCol.radius = danmakuPattern.radius;
        }
    );
}
```

Thank You!

Questions?